

VOLUME 10 / RUST BACKEND MASTERY

RustNote — CRUD・検索・UI 完全実装

一覧・作成・詳細・編集・完了・削除・検索・ページ送り・お知らせ・使いやすさを、1画面ずつ動かして確かめる

第10巻: RustNote — CRUD・検索・UI 完全実装

この巻で身に付けること

第9巻で、RustNote の要件・画面・データの決まり・フォルダの構成を設計しました。この巻では、その設計がノートの画面の1つずつで、どう**コードになっているか**を読み、動かして確かめます。

ノートの画面は、作る（Create）・読む（Read）・直す（Update）・消す（Delete）の4つの操作でできています。頭文字をとって **CRUD**（クラッド）と呼びます。どのアプリにもある基本の形ですが、作り方を1つまちがえると、次のような事故が起きます。

- 一覧に、他人のノートが混ざって表示される。あるいは件数だけが、他人の分まで数えられる
- 作成のフォームを送った直後に再読み込みすると、同じノートがもう1件できる
- 2つのタブで同じノートを直したら、先に保存した内容が黙って消える
- 「完了」のボタンを素早く2回押したら、表示と DB の値が食い違う
- 検索の欄に `%` を入れたら、全部のノートが出てくる

この巻では、RustNote の**完成したソース**（`Rust/rustnote/`）をお手本にして、画面ごとに「どんな事故を、どのコードで防いでいるか」を、テストを書いて確かめていきます。

到達目標

この巻を終えると、次のことができるようになります。

- 一覧の画面で、持ち主の条件・並び順・空のときの表示・ページ送りを、1つの SQL と1つの `view!` で作れる
- 作成・編集のフォームで、入力を検査し、失敗したら 400 で入力を残したまま出し直し、成功したら 303 で移る（PRG）形を作れる
- URL の ID（`/notes/{id}`）を型に変え、見つからない・他人のノートを、同じ 404 にできる
- 楽観的ロック（`version`）で、別のタブの更新を上書きしないようにし、完了の切り替えを1つの SQL で行える
- 検索・ページ送り・お知らせ（フラッシュ）・狭い画面・キーボードでの操作まで含めて、画面をテストで確かめられる

学習地図

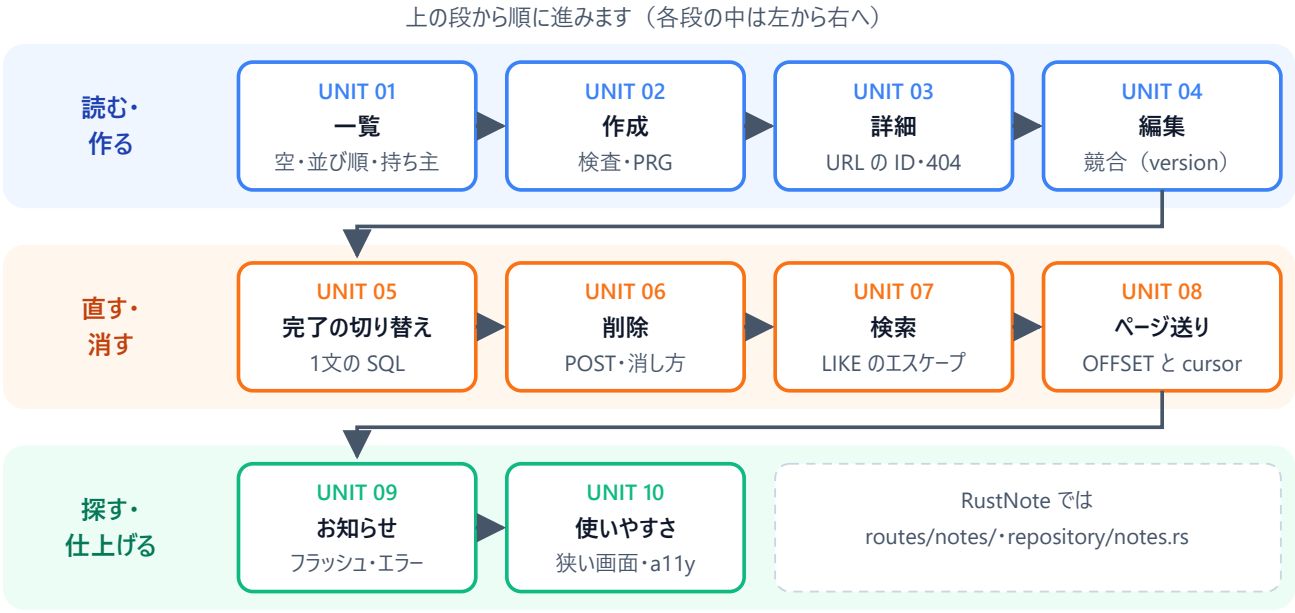


図10-1: 第10巻の学習地図。一覧と作成から始め、直す・消すを経て、検索・ページ送り・お知らせ・使いやすさで仕上げる

単元	テーマ	ひとことで言うと
UNIT 01	一覧・空のとき・持ち主の条件	自分のノートだけを、決まった順番で並べる
UNIT 02	作成のフォーム・入力の検査・PRG	失敗は 400 で出し直し、成功は 303 で移る
UNIT 03	詳細・URL の ID・見つからないとき（準備中）	無いノートも他人のノートも、同じ 404 にする
UNIT 04	編集・入力済みのフォーム・競合（準備中）	別のタブの更新を、黙って上書きしない
UNIT 05	完了の切り替え・1文の SQL（準備中）	読んでから書く形にせず、DB に1回で切り替えさせる
UNIT 06	削除・確かめ・消し方の選び方（準備中）	GET では消さない。消す前に確かめる
UNIT 07	検索・LIKE のエスケープ（準備中）	`%` や `_` を、ただの文字として探す
UNIT 08	ページ送り・OFFSET と cursor（準備中）	深いページでも、重ならず抜けずに送る
UNIT 09	お知らせ（フラッシュ）・エラーのまとめ（準備中）	移った先で一度だけ出し、エラーは同じ画面で見せる
UNIT 10	狭い画面・アクセシビリティ・総合の試験（準備中）	スマートフォンとキーボードでも使えるかを確かめる

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03〜10 は準備中で、次の更新から順に追加します。

前提知識（先に読んでおく巻）

- **第7巻（Turso・libSQL）**：`SELECT`・`WHERE`・`ORDER BY`・`LIMIT`（第7巻 UNIT 02）と、持ち主の条件を付けた SQL（第7巻 UNIT 03）。
- **第8巻（認証認可）UNIT 01:** 認証・セッション・認可の3つの確かめと、他人のノートを 404 にする理由。
- **第9巻（RustNote 設計）**：画面の一覧と状態（第9巻 UNIT 02）、ドメインの型（第9巻 UNIT 03）、モジュールの段と依存の向き（第9巻 UNIT 04）。この巻のテストも、第9巻と同じ形（`tests/` に足して `cargo test --test 名前`）で書きます。

> **注意:** 第6巻（Topcoat）は、書き直しの途中です。この巻で使う Topcoat の機能（`#[page]`・`#[component]`・`view!`・`href!`）は、本文の中で、そのつど説明します。

この巻の進め方

画面のコードは、ブラウザで開いて「動いた」で終わりにしがちです。各単元では、次の順番で進めてください。

1. **画面の約束を読む:** その画面の URL・返すステータスコード・出す文を、第9巻の画面の一覧で確かめます。
2. **コードを読む:** RustNote の `src/routes/` と `src/repository/notes.rs` の、その画面の部分を読みます。
3. **テストで確かめる:** 教材のテストを動かし、HTML の中身・並び順・DB の値を確かめます。
4. **1か所だけ外す:** 「よくあるエラーと読み方」の節で、守りを1つ外したときの様子を見ます。外したら、必ず元に戻します。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.98 以上**（本書シリーズの基準）。動作確認は rustc 1.99.0 / cargo 1.99.0（`aarch64-pc-windows-msvc`）
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では `src/` を変えずに、`tests/` と `examples/` に確かめ用のファイルを足す
topcoat	0.10.0（`#[page]`・`#[component]`・`view!`・`href!`）
libsql	0.10.0（ローカルの DB ファイル）

練習用のフォルダを用意する

第9巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる（1.99.0 になる）
rustup show active-toolchain
```

この巻のテストは、RustNote の `tests/common/mod.rs` にある道具（`test_app`・`sign_up`・`create_note`・`body_text` など）を使います。テストごとに一時フォルダに新しい DB を作り、ポートを開かずに、ルーターへ直接リクエストを渡して動かします（第9巻 UNIT 01）。

UNIT 01: 一覧・空のとき・持ち主の条件

この単元のゴール

- ノートの一覧（`GET /notes`）が、ログインの確かめ → URL のクエリの読み取り → DB の検索 → HTML の組み立て、の順で作られていることを説明できる
- 一覧の SQL に、持ち主の条件（`user_id = ?1`）と、並び順（`ORDER BY updated_at DESC, id DESC`）と、ページの区切り（`LIMIT`・`OFFSET`）が入っている理由を言える
- ノートが 0 件のときの文と、検索で 0 件のときの文を出し分ける `view!` を読み、テストで確かめられる
- 利用者が入力したタイトルが、HTML のタグとしてではなく文字として表示されること（自動エスケープ）を確かめられる

たとえ話でつかむ

マンションの、共同の宅配ボックスを考えます。ボックスには、住んでいる人全員の荷物が入っています。

- 管理人さんに「私の荷物を見せてください」と頼むと、管理人さんは、まず**部屋の鍵**を見て、本当にその部屋の人かを確かめます。
- 次に、**自分の部屋番号の札が付いた荷物だけ**を、棚から出してきます。隣の部屋の荷物は、たとえ同じ棚にあっても出しません。
- 出してきた荷物は、**新しく届いた順**に並べます。同じ時刻に届いた荷物は、受付の番号の大きいほうを先にします。こうすれば、何度頼んでも、毎回同じ順で並びます。
- 荷物が1つも無いときは、空の机を見せるのではなく「お荷物はありません」と伝えます。
- 荷物が多いときは、20 個ずつ机に並べて、「1 / 2（全 21 個）」と伝えます。

RustNote の一覧の画面も、同じ順番で作ります。

宅配ボックス	RustNote の一覧（GET /notes）
部屋の鍵を見て、その部屋の人かを確かめる	`require_user(cx)`（ログインの確かめ。第8巻 UNIT 01）
自分の部屋番号の札が付いた荷物だけを出す	SQL の `WHERE user_id = ?1`（持ち主の条件）
新しく届いた順、同じ時刻なら受付の番号の大きい順	`ORDER BY updated_at DESC, id DESC`
20 個ずつ並べる	`LIMIT 20 OFFSET …`（`PER_PAGE = 20`）
「お荷物はありません」	`if page.notes.is_empty()` の文
「1 / 2（全 21 個）」	ページ送りの部品（`pagination`）と、件数の SQL（`COUNT(*)`）

> **用語:** CRUD（クラッド：Create（作る）・Read（読む）・Update（直す）・Delete（消す）の頭文字。データを扱う画面の基本の4つの操作）

> **用語:** 空の状態（からのじょうたい。empty state：表示するデータが1件も無いときの画面。何も表示しないのではなく、次にすること（「最初の1件を書いてみましょう」）を伝える）

図で理解する

`GET /notes` のリクエストが届いてから、HTML を返すまでの流れです。

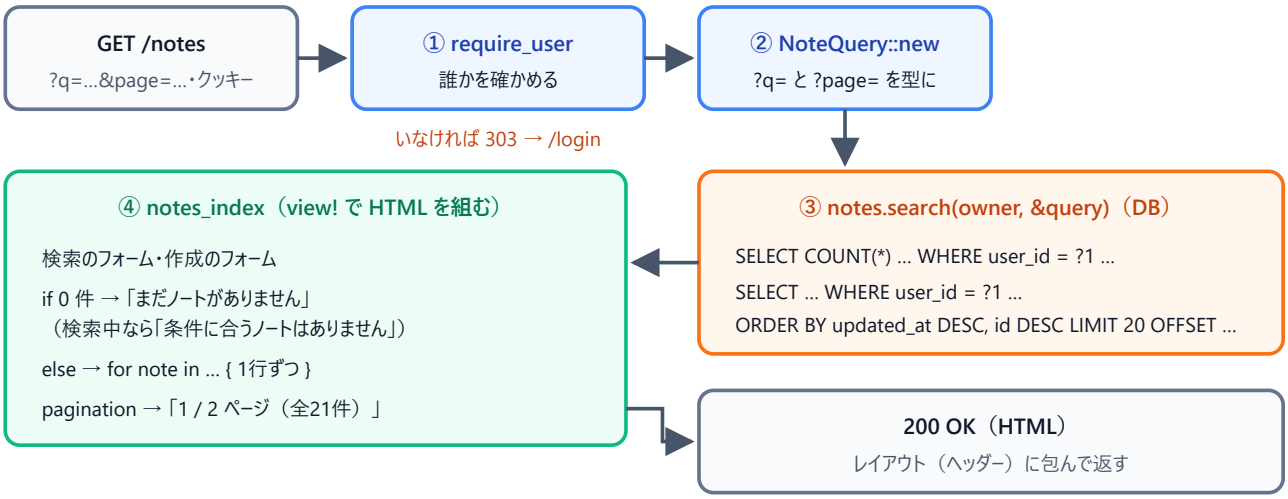


図10-2: 一覧の画面ができるまで。誰かを確かめ、URL のクエリを型にし、持ち主の条件付きの SQL で数えて読み、view! で HTML にする

- ① と ② は、ページの関数 `index` の中です。`require_user(cx)` でログイン中のユーザーを確かめ、`query_params` で URL の `?q=`（キーワード）と `?page=`（ページ番号）を読み、`NoteQuery::new` で検査済みの型にします。
- ③ は、リポジトリの `search` です。件数を数える SQL と、そのページの分を読む SQL の2つを送ります。どちらにも `user_id = ?1` が入っています（問2で、片方だけ忘れたらどうなるかを確かめます）。
- ④ は、コンポーネントの `notes_index` です。読んだノートが 0 件か、そうでないかで、表示を分けます。

並び順の決まりは、次の図のとおりです。

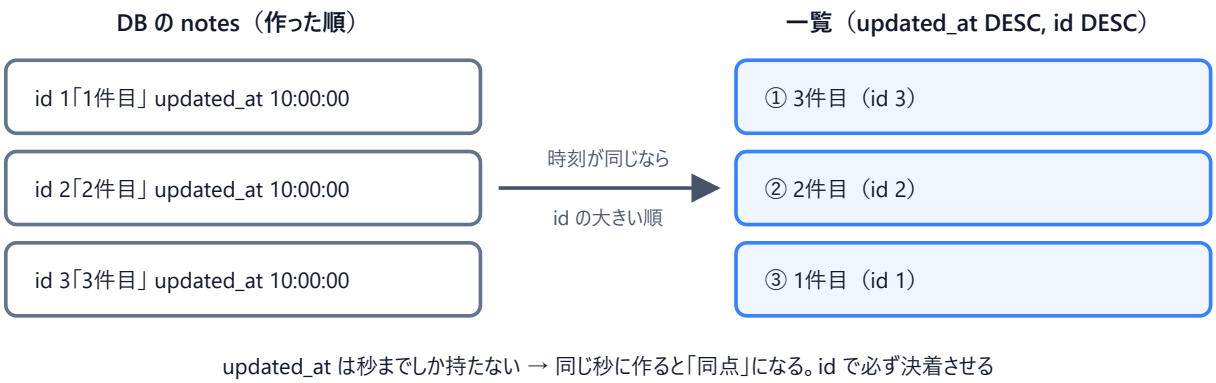


図10-3: 一覧の並び順。最後に更新した時刻（updated_at）の新しい順に並べ、同じ時刻のときは id の大きい順にする。id は重ならないので、順番が必ず1つに決まる

- `updated_at` は、`datetime('now')` で入れた「秒まで」の時刻です。テストのように速く続けて作ると、3件とも同じ時刻になります。
- そこで、2つ目の並べ方として `id DESC` を足しています。`id` は1件ごとに違う数なので、同じ時刻のノートも、必ず決まった順番に並びます。

まず動かす最小コード

例1-1: 一覧の画面を、3つの場面で確かめる

> **▼ このコードがやること（先に日本語で）**：3つのテストで、一覧の画面（ `GET /notes` ）の HTML を調べます。1つ目は、ノートが 0 件のときの文、3件作ったときの並び順とページ送りの文、bob から見た一覧です。2つ目は、3件目の更新の時刻だけを1日前にずらしたときの並び順です。3つ目は、タグを含むタイトルが、文字として表示されるかです。

```rust
//! 第10巻 UNIT 01: ノートの一覧（GET /notes）の、空のとき・並び順・持ち主ごとの表示を確かめる。
//! 実行: cargo test --test u01_list -- --nocapture --test-threads=1

mod common;

use common::{TestApp, body_text, test_app};

const PASSWORD: &str = "correct horse battery";

/// 一覧の HTML に、どのタイトルが、どの順番で出ているか
fn order<'a>(html: &str, titles: &[&'a str]) -> Vec<&'a str> {
    let mut found: Vec<(usize, &str)> = titles
        .iter()
        .filter_map(|title| html.find(title).map(|at| (at, *title)))
        .collect();
    found.sort();
    found.into_iter().map(|(_, title)| title).collect()
}

/// 一覧を開いて、HTML を返す
async fn list(app: &TestApp, cookie: &str) -> String {
    body_text(app.get("/notes", Some(cookie)).await).await
}

#[tokio::test]
async fn empty_then_newest_first() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;

    let html = list(&app, &alice).await;
    println!(
        "[ノートが 0 件] 空の文が出る: {}",
        html.contains("まだノートがありません")
    );

    for title in ["1件目", "2件目", "3件目"] {
        app.create_note(&alice, title, "").await;
    }
    let html = list(&app, &alice).await;
    println!(
        "[3 件作った] 並び順: {:?}",
        order(&html, &["1件目", "2件目", "3件目"])
    );
    println!(
        "[3 件作った] ページ送り: {}",
        html.contains("1 / 1 ページ（全3件）")
    );

    let bob = app.sign_up("bob@example.com", PASSWORD).await;
```

```
    let html = list(&app, &bob).await;
    println!(
        "[bob の一覧] 空の文が出る: {} / alice のノートが出る: {}",
        html.contains("まだノートがありません"),
        html.contains("1件目")
    );
    println!(
        "[bob の一覧] ページ送り: {}",
        html.contains("1 / 1 ページ（全0件）")
    );
}

#[tokio::test]
async fn ordered_by_updated_at_then_id() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;
    for title in ["1件目", "2件目", "3件目"] {
        app.create_note(&alice, title, "").await;
    }

    // 3件目の「最後に更新した時刻」を、1日前にずらす
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    conn.execute(
        "UPDATE notes SET updated_at = datetime('now', '-1 day') WHERE title = '3件目'",
        (),
    )
    .await
    .expect("更新できません");

    let html = list(&app, &alice).await;
    println!(
        "[3件目を 1 日前の更新にした] 並び順: {:?}",
        order(&html, &["1件目", "2件目", "3件目"])
    );
}

#[tokio::test]
async fn titles_are_escaped() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;
    app.create_note(&alice, "<b>太字</b> & <script>", "").await;

    let html = list(&app, &alice).await;
    println!("タグのまま入っている: {}", html.contains("<b>太字</b>"));
    println!(
        "文字に置き換わっている: {}",
        html.contains("&lt;b&gt;太字&lt;/b&gt; &amp; &lt;script&gt;")
    );
}
```

コピーした `rustnote` フォルダの `tests/u01_list.rs` として保存して実行します。

```
cargo test --test u01_list -- --nocapture --test-threads=1
```

▼ 実行結果

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 19.78s
     Running tests\u01_list.rs (target\debug\deps\u01_list-d04e2970d522cf0b.exe)

running 3 tests
test empty_then_newest_first ... POST /signup -> 303 See Other (466.6657ms)
GET /notes -> 200 OK (9.5072ms)
[ノートが 0 件] 空の文が出る: true
POST /notes -> 303 See Other (9.6777ms)
POST /notes -> 303 See Other (11.3172ms)
POST /notes -> 303 See Other (7.7175ms)
GET /notes -> 200 OK (5.0939ms)
[3 件作った] 並び順: ["3件目", "2件目", "1件目"]
[3 件作った] ページ送り: true
POST /signup -> 303 See Other (396.9523ms)
GET /notes -> 200 OK (3.432ms)
[bob の一覧] 空の文が出る: true / alice のノートが出る: false
[bob の一覧] ページ送り: true
ok
test ordered_by_updated_at_then_id ... POST /signup -> 303 See Other (426.5175ms)
POST /notes -> 303 See Other (6.6846ms)
POST /notes -> 303 See Other (7.1091ms)
POST /notes -> 303 See Other (6.2833ms)
GET /notes -> 200 OK (8.3724ms)
[3件目を 1 日前の更新にした] 並び順: ["2件目", "1件目", "3件目"]
ok
test titles_are_escaped ... POST /signup -> 303 See Other (427.4357ms)
POST /notes -> 303 See Other (5.3111ms)
GET /notes -> 200 OK (3.8569ms)
タグのまま入っている: false
文字に置き換わっている: true
ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
1.87s
```

▼ コードを1つずつ分解して解説

解説1: 0 件のときは、空の文を出す

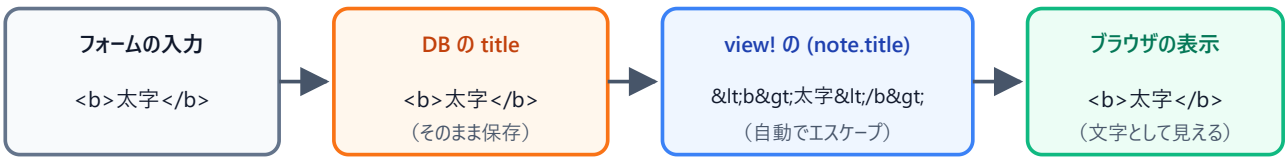
- ノートが 0 件のとき、一覧には「まだノートがありません。最初の1件を書いてみましょう。」が出ました（ `空の文が出る: true` ）。何も表示しない画面だと、利用者は「読み込みに失敗したのか」「本当に 0 件なのか」を区別できません。次にすることまで書いておくのが、空の状態の作り方です。
- bob の一覧も、空の文が出て、alice のノート（ `1件目` ）は出ませんでした。ページ送りも `1 / 1 ページ（全0件）` で、alice の 3 件は数えられていません。

解説2: 新しい順、同じ時刻なら id の大きい順

- 3件を続けて作ると、並び順は `["3件目", "2件目", "1件目"]` でした。3件は同じ秒に作られたので、 `updated_at` は同じです。 `id DESC` で、あとから作った（id の大きい）ノートが上に来ました。
- 3件目の `updated_at` を1日前にずらすと、並び順は `["2件目", "1件目", "3件目"]` になりました。 `updated_at` が古い 3件目は、id が大きくても下に移ります。並び順の1つ目の決まりは `updated_at` 、2つ目が `id` です。
- `order` は、テストのための小さな道具です。HTML の中で、それぞれのタイトルが何文字目に出てくるかを調べ、前から順に並べ直しています。

解説3: タイトルは、タグではなく文字として出る

- タイトルに `<b>太字</b> & <script>` を入れると、HTML の中には `<b>太字</b> & <script>` と書かれていました（ `タグのまま入っている: false` ）。 `<` が `<` 、 `>` が `>` 、 `&` が `&` に置き換わっています。ブラウザには、 `<b>太字</b>` という**文字**が、そのまま表示されます。
- これは、RustNote の `view!` の `(note.title)` のように、 `( )` で値を入れた所が、自動で**エスケープ**（HTML で特別な意味を持つ文字を、ただの文字の書き方に置き換えること）されるからです。利用者が `<script>` を書いても、プログラムとして動きません（ `src/views.rs` の先頭のコメント）。



保存するときは入力のまま、HTML にするときにエスケープする（太字にはならず、<script> も動かない）

図10-4: タイトルにタグを入れたときの流れ。DB には入力のまま保存し、view! が HTML にするときに置き換えるので、ブラウザでは文字として表示される

> **用語**: エスケープ（escape：特別な意味を持つ文字を、ただの文字として扱われる書き方に置き換えること。HTML では `<` を `<` にする）

もう一歩: 実務に近い形

例1-2: 並べ方に id を足さないと、どうなるか

`ORDER BY updated_at DESC` だけでは、同じ時刻のノートの順番は決まりません。本当にそうなるのか、DB に直接 SQL を送って確かめます。

> ▼ **このコードがやること（先に日本語で）**：一時フォルダの DB に、同じ秒のうちに3件のノートを入れます。`updated_at` の値を表示してから、`ORDER BY updated_at DESC` だけの SQL と、`, id DESC` を足した SQL で、タイトルを並べて表示します。

```rust
//! 並び替えに updated_at だけを使うと、同じ秒に更新したノートの順番が決まらないことを確かめる。
//! 実行: cargo run -q --example u01_order

use libsql::Connection;
use rustnote::db::{self, Db};

/// SQL を実行して、タイトルを順に並べる
async fn titles(conn: &Connection, sql: &str) -> anyhow::Result<Vec<String>> {
    let mut rows = conn.query(sql, ()).await?;
    let mut titles = Vec::new();
    while let Some(row) = rows.next().await? {
        titles.push(row.get::<String>(0)?);
    }
    Ok(titles)
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("u01.db")).await?;
    db::migrate(&db).await?;
    let conn = db.connect().await?;

    // 3件を、同じ秒のうちに続けて作る（updated_at は秒までしか持たない）
    conn.execute_batch(
        "INSERT INTO users (email, password_hash) VALUES ('alice@example.com', 'x');
         INSERT INTO notes (user_id, title) VALUES (1, '1件目');
         INSERT INTO notes (user_id, title) VALUES (1, '2件目');
         INSERT INTO notes (user_id, title) VALUES (1, '3件目');",
    )
    .await?;
    println!(
        "updated_at: {:?}",
        titles(&conn, "SELECT DISTINCT updated_at FROM notes").await?
    );

    let by_time = titles(
        &conn,
        "SELECT title FROM notes WHERE user_id = 1 ORDER BY updated_at DESC",
    )
    .await?;
    println!("ORDER BY updated_at DESC         → {by_time:?}");

    let by_time_and_id = titles(
        &conn,
        "SELECT title FROM notes WHERE user_id = 1 ORDER BY updated_at DESC, id DESC",
    )
    .await?;
    println!("ORDER BY updated_at DESC, id DESC → {by_time_and_id:?}");
```

```
    Ok(())
}
```

```
cargo run -q --example u01_order
```

▼ 実行結果

```
updated_at: ["2026-10-27 15:12:41"]
ORDER BY updated_at DESC          → ["3件目", "2件目", "1件目"]
ORDER BY updated_at DESC, id DESC → ["3件目", "2件目", "1件目"]
```

- 3件の `updated_at` は、どれも同じ秒でした。そして、この実験では、`id DESC` を足しても足さなくても、同じ `["3件目", "2件目", "1件目"]` になりました。
- では、`id DESC` は要らないのでしょうか。いいえ、要ります。SQL の決まりでは、`ORDER BY` の値が同じ行どうしの順番は**決まっていません**。この実験で同じ順になったのは、今の SQLite が、同じ値の行を、たまたま行の番号（id）の大きい順に返したからです。索引を足したり、DB を Turso に変えたり、SQLite の版が上がったりしたときに、同じ順になる保証はありません。
- 並び順が決まっていないと、ページ送りで困ります。1ページ目と2ページ目で同点の行の順番が入れ替わると、同じノートが2つのページに出たり、どのページにも出なかったりします。だから、**最後に、重ならない列（`id`）を足して、順番を必ず1つに決める**のです。結果が同じでも、「たまたま」ではなく「決まり」で同じにしておきます。

> **注意:** 「試したら同じ順だったから大丈夫」と考えるのは危険です。テストは、今の DB の作りでの1回の結果しか確かめられません。SQL の決まりで保証されていないことは、コードで保証します。

よくあるエラーと読み方

エラー1: for で回したあとに、同じ値を使った（E0382）

`notes_index` の `view!` の中では、ノートを `for note in page.notes.clone()` で1行ずつ並べています。`.clone()`（複製）は無駄に見えるので、外してみた場合です。試すときは、練習用のコピーの `src/routes/notes.rs` を書き換え、確かめたら必ず元に戻してください。

```
                for note in page.notes {
```

```
cargo build
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0382]: use of partially moved value
   --> src\routes\notes.rs:104:26
    |
 89 |                 for note in page.notes {
    |                             ---------- value partially moved due to this implicit
call to `.into_iter()`
...
104 |         pagination(page: page, query: query)
    |                          ^^^^ value used here after partial move
    |
note: the `for` loop is desugared into a call to `std::iter::IntoIterator::into_iter`,
which takes ownership of the receiver `self`, which moves value
   --> 
/rustc/b940084d7eb6a299eb4bfeb8e34901bc051e7ac4/library\core\src\iter\traits\collect.rs
:312:17
    = note: partial move occurs because value has type `std::vec::Vec<Note>`, which
does not implement the `Copy` trait
help: consider iterating over a slice of the `std::vec::Vec<Note>`'s content to avoid
moving into the `for` loop
    |
 89 |                 for note in &page.notes {
    |                             +

For more information about this error, try `rustc --explain E0382`.
error: could not compile `rustnote` (lib) due to 1 previous error
```

- **何が起きたか:**「一部が移動した値を使いました」というエラーです。89 行目の `for note in page.notes` で、`page` の中の `notes`（`Vec<Note>`）が for の中へ**移動**しました（`note:` が、for は `into_iter()` を呼んで、値の持ち主を移す、と説明しています）。そのあと、104 行目の `pagination(page: page, ...)` で `page` を丸ごと渡そうとしたので、「`notes` はもう `page` の中に無い」と止められました（第3巻）。
- **直し方:** RustNote のとおり `page.notes.clone()` で、ノートの一覧を複製してから for に渡します。`page` の中の `notes` は残るので、ページ送りの部品にも `page` を丸ごと渡せます。

エラー2: help の案内どおりに & を付けた（E0507・E0505）

エラー1の `help:` は、「`&page.notes` のように、借りて回してはどうか」と案内しています。案内どおりに書き換えた場合です。

```
                for note in &page.notes {
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0507]: cannot move out of `note.title` which is behind a shared reference
...
Some errors have detailed explanations: E0505, E0507.
For more information about an error, try `rustc --explain E0505`.
error: could not compile `rustnote` (lib) due to 3 previous errors
```

（エラーの全文は長いので、最初の2行と最後の3行だけを載せています。間に、`note.updated_at` について同じ形の `E0507` と、`E0505` が続きます）

- **何が起きたか:** 今度は「共有の参照の後ろにある `note.title` からは、値を移動できません」というエラーが、`note.title` と `note.updated_at` で2つ、さらに `E0505`（借りている値を移動しようとした）が1つ出ました。Topcoat の `view!` は、`(note.title)` のように入れた値を**持ち主ごと受け取って** HTML にするので、借りただけの `note` からは渡せないのです。
- **学べること:** コンパイラーの `help:` は、そのエラーを消すための1つの案です。プログラム全体の作り（ここでは、`view!` が値を受け取る形）までは見ていないので、案内どおりに直すと、別のエラーになることがあります。エラーを1つずつ読んで、**なぜ移動が起きるのか**を考えるのが近道です。
- **どちらを選ぶか:** 1ページに並ぶのは多くても 20 件なので、`clone()` で複製しても、速さはほとんど変わりません。読みやすさを優先して、RustNote は `clone()` にしています。確かめたら、`src/routes/notes.rs` を元に戻します。

RustNote ではここで使う

一覧のページの関数 `index` です（`src/routes/notes.rs` から抜き出しています）。

```
#[page]
pub async fn index(cx: &Cx) -> Result<impl View> {
    let user = require_user(cx).await?;
    let params = query_params::<NotesParams>(cx)?;
    let query = NoteQuery::new(params.q.as_deref(), params.page);
    let flash = flash::take(cx);

    Ok(view! {
        flash_banner(flash: flash)
        notes_index(owner: user.id, query: query, form: NoteFormState::default())
    })
}
```

- 最初の行で `require_user(cx)` を呼びます。ログインしていなければ、ここで 303 になり、下の行は動きません。
- `notes_index` に渡すのは、`owner: user.id` です。一覧の部品は、持ち主を引数で受け取るので、「誰の一覧か」を、部品の中で勝手に決めることができません。

ノートを1行ずつ並べるところです（`notes_index` の `view!` から抜き出しています）。

```
        if page.notes.is_empty() {
            <p class="empty">
                if keyword.is_some() {
                    "条件に合うノートはありません。"
                } else {
                    "まだノートがありません。最初の1件を書いてみましょう。"
                }
            </p>
        } else {
            <ul class="note-list">
                #[key(note.id.get())]
                for note in page.notes.clone() {
                    <li class=(class!("note-item", "done" if note.done))>
                        <a href=(href!(id::show, id::Id(note.id)))>(note.title)</a>
                        if note.done {
                            <span class="badge">"完了"</span>
                        }
                        <span class="updated">
                            "更新: "
                            (note.updated_at)
                        </span>
                    </li>
                }
            </ul>
        }
```

- `view!` の中には、Rust の `if` と `for` を、そのまま書けます。`#[key(note.id.get())]` は、1行ずつに「このノートの番号」という目印を付ける指定です。
- `class!("note-item", "done" if note.done)` は、完了したノートだけに `done` のクラスを足します（問1）。
- リンク先は `href!(id::show, id::Id(note.id))` で作ります。URL を文字列で `"/notes/" + id` と組み立てるのではなく、詳細のページの関数（`id::show`）と、その ID の型を指定するので、ページを消したり名前を変えたりすると、コンパイルエラーで気づけます（第9巻 UNIT 04）。

DB を検索するところは、図の③で見た SQL です（`src/repository/notes.rs` から抜き出しています）。

```
        // LIKE の中の \ を「次の文字をそのまま扱う」という意味にする（ESCAPE '\'）
        let condition = "user_id = ?1 AND (title LIKE ?2 ESCAPE '\\' OR body LIKE ?2
ESCAPE '\\')";

        // 1) 条件に合う件数（ページ数の計算に使う）
        let count_sql = format!("SELECT COUNT(*) FROM notes WHERE {condition}");
```

```
    // 2) このページに表示する分（新しく更新した順）
    let list_sql = format!(
        "SELECT {NOTE_COLUMNS} FROM notes WHERE {condition}
         ORDER BY updated_at DESC, id DESC
         LIMIT ?3 OFFSET ?4"
    );
```

- 持ち主の条件を含む `condition` を、件数の SQL と一覧の SQL の**両方で使い回して**います。条件を1か所に書くので、片方だけ書き忘れる心配がありません（問2）。
- キーワードが無いときは、`?2` に `%`（何にでも一致する）を入れます。キーワードの中の `%` や `_` をただの文字として扱う `escape_like` は、UNIT 07（準備中）で扱います。

練習問題

問1: 3件のノートを作り、3件とも「最後に更新したのは1時間前」にしてから、1件目だけを完了にします（`POST /notes/1/toggle`）。一覧の並び順と、完了の印（`class="note-item done"` と「完了」のバッジ）は、どうなるでしょうか。予想してから、テストで確かめてください。

問2: もし、件数を数える SQL にだけ持ち主の条件を書き忘れたら、alice が 3 件、bob が 0 件のとき、bob の一覧のページ送りには何と表示されるでしょうか。DB に直接 SQL を送る小さなプログラムで確かめてください。

問3: RustNote の一覧は、ノートが 0 件のときと、検索で 0 件のときで、出す文を変えています。それぞれ、どんな文が出るかを確かめてください。また、文を分けている理由を説明してください。

解答と解説

問1の解答:

`tests/u01_q1.rs` です。

```rust
//! 問1の解答: 完了にしたノートは、一覧でどう表示され、どこに並ぶか。
//! 実行: cargo test --test u01_q1 -- --nocapture

mod common;

use common::{body_text, test_app};

#[tokio::test]
async fn toggled_note_moves_to_top_with_badge() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;
    let first = app.create_note(&alice, "1件目", "").await;
    app.create_note(&alice, "2件目", "").await;
    app.create_note(&alice, "3件目", "").await;

    // 3件とも、最後に更新したのは1時間前だったことにする
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    conn.execute(
        "UPDATE notes SET updated_at = datetime('now', '-1 hour')",
        (),
    )
    .await
    .expect("更新できません");

    // 1件目を完了にする（更新の時刻が今になる）
    app.post_form(&format!("{first}/toggle"), "", Some(&alice))
        .await;

    let html = body_text(app.get("/notes", Some(&alice)).await).await;
    let mut found: Vec<(usize, &str)> = ["1件目", "2件目", "3件目"]
        .into_iter()
        .filter_map(|title| html.find(title).map(|at| (at, title)))
        .collect();
    found.sort();
    let order: Vec<&str> = found.into_iter().map(|(_, title)| title).collect();
    println!("並び順: {order:?}");
    println!(
        "完了の印の付いた行: {}",
        html.matches(r#"class="note-item done""#).count()
    );
    println!(
        "「完了」のバッジ: {}",
        html.matches(r#"<span class="badge">完了</span>"#).count()
    );
}
```

```
cargo test --test u01_q1 -- --nocapture
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 13.14s
     Running tests\u01_q1.rs (target\debug\deps\u01_q1-81fa10c5a5eabbd3.exe)

running 1 test
POST /signup -> 303 See Other (415.3854ms)
POST /notes -> 303 See Other (6.5261ms)
POST /notes -> 303 See Other (6.1675ms)
POST /notes -> 303 See Other (8.6278ms)
POST /notes/1/toggle -> 303 See Other (16.379ms)
GET /notes -> 200 OK (14.0618ms)
並び順: ["1件目", "3件目", "2件目"]
完了の印の付いた行: 1
「完了」のバッジ: 1
test toggled_note_moves_to_top_with_badge ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.50s
```

- 1件目がいちばん上に移りました。完了の切り替え（`toggle_done`）の SQL は、`done` を反対にするのと一緒に、`updated_at = datetime('now')` で更新の時刻を今にするからです（`src/repository/notes.rs`）。残りの2件は、どちらも1時間前の同じ時刻なので、`id DESC` で 3件目、2件目の順です。
- 完了の印の付いた行と「完了」のバッジは、それぞれ 1 つずつでした。完了にしたのは1件目だけだからです。
- 「完了にしただけで、一覧の上に移る」のが使いやすいかどうかは、アプリの決めごとです。完了したノートを下に集めたいなら、`ORDER BY done, updated_at DESC, id DESC` のように、`done`（0 が未完了、1 が完了）を最初に並べます。

問2の解答:

`examples/u01_q2.rs` です。

```rust
//! 問2の解答: 件数を数える SQL にだけ、持ち主の条件を書き忘れたら、bob の一覧はどうなるか。
//! 実行: cargo run -q --example u01_q2

use libsql::{Connection, params::IntoParams};
use rustnote::db::{self, Db};

/// 1つの数を返す SQL を実行する
async fn count(conn: &Connection, sql: &str, params: impl IntoParams) ->
anyhow::Result<i64> {
    let mut rows = conn.query(sql, params).await?;
    let row = rows.next().await?.expect("行があるはずです");
    Ok(row.get::<i64>(0)?)
}

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("u01.db")).await?;
    db::migrate(&db).await?;
    let conn = db.connect().await?;
    conn.execute_batch(
        "INSERT INTO users (email, password_hash)
             VALUES ('alice@example.com', 'x'), ('bob@example.com', 'x');
         INSERT INTO notes (user_id, title) VALUES (1, '1件目'), (1, '2件目'), (1, '3件
目');",
    )
    .await?;
    let bob: i64 = 2;

    // 一覧に並べる SQL（RustNote と同じく、持ち主の条件がある）
    let listed = count(
        &conn,
        "SELECT COUNT(*) FROM (SELECT id FROM notes WHERE user_id = ?1 LIMIT 20)",
        [bob],
    )
    .await?;
    // 間違い: 件数の SQL に、持ち主の条件を書き忘れた
    let wrong = count(
        &conn,
        "SELECT COUNT(*) FROM notes WHERE title LIKE '%' OR body LIKE '%'",
        (),
    )
    .await?;
    // RustNote と同じ: 件数の SQL にも、持ち主の条件を入れる
    let right = count(
        &conn,
        "SELECT COUNT(*) FROM notes WHERE user_id = ?1 AND (title LIKE '%' OR body LIKE
'%')",
        [bob],
```

```
    )
    .await?;
    println!("bob の一覧に並ぶノート: {listed} 件");
    println!("件数の SQL に持ち主の条件が無い: 1 / 1 ページ（全{wrong}件）");
    println!("件数の SQL にも持ち主の条件がある: 1 / 1 ページ（全{right}件）");
    Ok(())
}
```

```
cargo run -q --example u01_q2
```

```
bob の一覧に並ぶノート: 0 件
件数の SQL に持ち主の条件が無い: 1 / 1 ページ（全3件）
件数の SQL にも持ち主の条件がある: 1 / 1 ページ（全0件）
```

- bob の一覧に並ぶノートは 0 件なのに、件数の SQL に持ち主の条件が無いと、ページ送りには `1 / 1 ページ（全3件）` と出ます。画面には何も並んでいないのに「全3件」と出るので、利用者は混乱します。
- それだけではありません。bob は、件数の数字から「ほかの人が全部で何件ノートを書いているか」を知ることができます。ノートが 21 件を超えると、「次へ」のリンクまで出て、押すと空のページが出ます。**件数も、他人のデータの一部**です。
- RustNote は、持ち主の条件を `condition` という1つの文字列にまとめ、2つの SQL で使い回しているので、この間違いが起きにくくなっています。

問3の解答:

`tests/u01_q3.rs` です。

```rust
//! 問3の解答: ノートが 0 件のときと、検索で 0 件のときで、出る文が違うことを確かめる。
//! 実行: cargo test --test u01_q3 -- --nocapture

mod common;

use common::{body_text, test_app};

#[tokio::test]
async fn empty_messages_differ() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;
    app.create_note(&alice, "買い物", "牛乳").await;

    for url in ["/notes", "/notes?q=宿題"] {
        let html = body_text(app.get(url, Some(&alice)).await).await;
        let empty = html
            .split(r#"<p class="empty">"#)
            .nth(1)
            .and_then(|rest| rest.split("</p>").next())
            .unwrap_or("（空の文は無し）");
        println!("{url} → {empty}");
    }

    let bob = app
        .sign_up("bob@example.com", "correct horse battery")
        .await;
    let html = body_text(app.get("/notes", Some(&bob)).await).await;
    let empty = html
        .split(r#"<p class="empty">"#)
        .nth(1)
        .and_then(|rest| rest.split("</p>").next())
        .unwrap_or("（空の文は無し）");
    println!("/notes（bob） → {empty}");
}
```

```
cargo test --test u01_q3 -- --nocapture
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.51s
     Running tests\u01_q3.rs (target\debug\deps\u01_q3-f9715f282720c475.exe)

running 1 test
POST /signup -> 303 See Other (439.0876ms)
POST /notes -> 303 See Other (6.562ms)
GET /notes -> 200 OK (4.1019ms)
/notes → （空の文は無し）
GET /notes -> 200 OK (3.3416ms)
/notes?q=宿題 → 条件に合うノートはありません。
POST /signup -> 303 See Other (422.86ms)
GET /notes -> 200 OK (2.7616ms)
/notes（bob）→ まだノートがありません。最初の1件を書いてみましょう。
test empty_messages_differ ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.90s
```

- alice は「買い物」のノートを持っているので、`/notes` には空の文が出ません。「宿題」で検索すると「条件に合うノートはありません。」、ノートを1件も持っていない bob には「まだノートがありません。最初の1件を書いてみましょう。」が出ました。
- **文を分ける理由:** 検索で 0 件のときに「まだノートがありません」と出すと、利用者は「自分のノートが消えた」と驚きます。反対に、本当に 0 件の人に「条件に合うノートはありません」と出すと、何を直せばよいのか分かりません。**0 件になった理由ごとに、次にすること**（キーワードを変える・最初の1件を書く）を伝えるのです。`view!` の中の `if keyword.is_some()` が、この出し分けをしています（第9巻 UNIT 02 の画面の状態）。

まとめ

- 一覧（`GET /notes`）は、`require_user` で誰かを確かめ、URL のクエリを `NoteQuery` にし、持ち主の条件付きの2つの SQL（件数と、そのページの分）で読み、`view!` で HTML にする。持ち主の条件は1か所に書いて、2つの SQL で使い回す。
- 並び順は `ORDER BY updated_at DESC, id DESC`。同じ時刻の行の順番は SQL では決まっていないので、試して同じでも、最後に重ならない `id` を足して決める。
- 0 件のときは、理由ごとに次にすることを伝える文を出す。`view!` に `( )` で入れた値は自動でエスケープされ、利用者のタイトルはタグではなく文字として表示される。

UNIT 02: 作成のフォーム・入力の検査・PRG

この単元のゴール

- ノートの作成（`POST /notes`）が、ログインの確かめ → 入力の検査 → 保存 → 303 で詳細へ、の順で動くことを説明できる
- 入力に問題があったとき、400 で**同じ画面**を出し直し、書いた内容を消さずに、項目ごとのエラーを全部まとめて見せる理由を言える
- PRG（Post/Redirect/Get）で、再読み込みしてもノートが二重にできないことを、テストで確かめられる。二重送信まで防げるわけではないことも言える
- フォームの `name` と `NoteForm` の項目名が合わないときの 400 と、`String` と `&str` を取り違えたときの `E0308` を読める

たとえ話でつかむ

役所の窓口に、申請書を出す場面を考えます。

- 申請書に**書き漏れ**があると、窓口の人は受け付けません。申請書を**そのまま返して**、直す所に付箋を貼ります。書いてあった所は消しません。付箋は、気づいた所に**全部**貼ります（1つ直して出し直したら、また別の所を言われる、とならないように）。
- 申請書がそろっていれば受け付けて、「**3番の窓口で控えを受け取ってください**」と案内します。窓口の人は、ここで控えを渡しません。
- 3番の窓口で控えを受け取ると、控えには「受け付けました」のスタンプが**一度だけ**押してあります。
- 控えを**もう一度見に行っても**、申請が2件になることはありません。ところが、同じ申請書を**もう一度窓口に出せば**、申請は2件になります。

役所の窓口	RustNote の作成（POST /notes）
申請書を出す	フォームを送る（`POST /notes`）
書き漏れがあれば、申請書を返して付箋を貼る	400 で、入力を残したフォームと、項目ごとのエラーを出し直す
付箋は全部まとめて貼る	`NewNote::parse` が、タイトルと本文の問題を両方集める
受け付けたら、控えは別の窓口で	303 と `Location: /notes/1`（ブラウザが、改めて GET で取りに行く）
スタンプは一度だけ	お知らせ（フラッシュ）のクッキー。表示したら消す
控えを見直す	再読み込み（GET をもう一度）。ノートは増えない
申請書を出し直す	同じ POST をもう一度送る。ノートは増える

> **用語:** PRG（ピーアールジー。Post/Redirect/Get：フォームの POST が成功したら、画面を直接返さずに、303 で別の URL へ移し、ブラウザに GET で開き直させる形。第9巻 UNIT 02）

> **用語:** 入力の検査（バリデーション：送られてきた値が、決まり（タイトルは 1〜80 文字など）に合っているかを確かめること。合わなければ、保存せずに断る）

> **用語:** 400 Bad Request（「送られてきた内容に問題がある」という返事。利用者が直せば通る失敗に使う）

図で理解する

`POST /notes` が届いてから、返事をするまでの流れです。

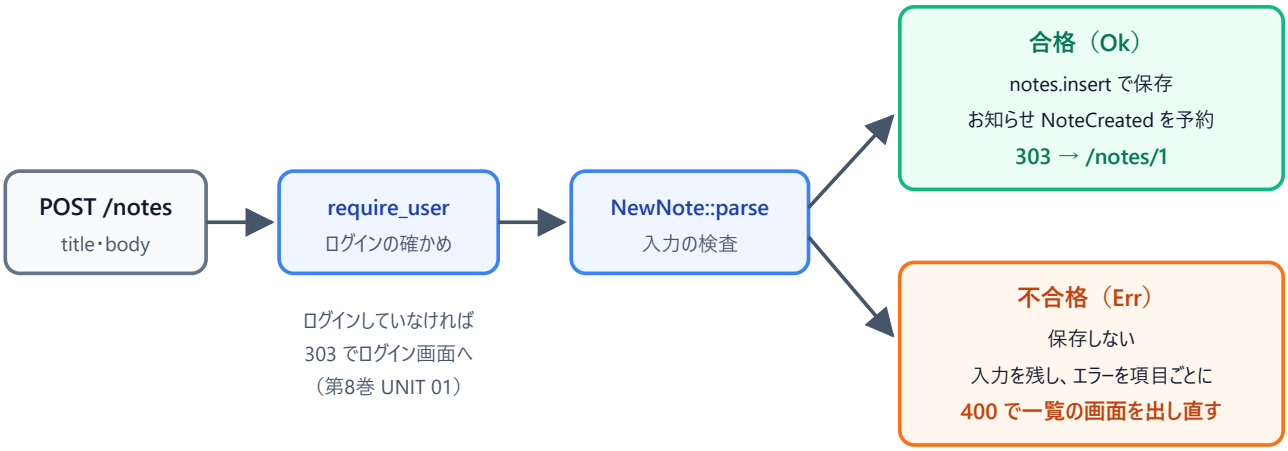


図10-5: POST /notes の流れ。検査に合格したときだけ保存して 303 で移り、不合格なら保存せずに 400 で同じ画面を出し直す

成功したときの、ブラウザとサーバーのやり取りです。



図10-6: 成功したときの PRG。POST の返事は 303 と移動先だけで、画面は GET で取りに行く。再読み込みでくり返されるのは GET なので、ノートは二重にできない

- ②の 303 は「次は、この URL を **GET で**取りに来てください」という返事です。本文の画面はありません。
- ブラウザは③で、自分で `/notes/1` を GET します。アドレス欄も `/notes/1` に変わります。だから⑤の再読み込みは「GET /notes/1 をもう一度」になり、ノートは増えません。
- お知らせ（「ノートを作成しました」）は、クッキーで②から③へ受け渡し、④で表示したら消します。これで、⑥ではもう出ません（お知らせのくわしい仕組みは UNIT 09（準備中））。

検査は、タイトルと本文を**両方とも**調べてから、結果をまとめます。



図10-7: NewNote::parse の4つの結果。両方とも合格のときだけ NewNote ができ、どちらかが不合格なら、見つかった問題を NoteInputErrors に全部入れて返す

まず動かす最小コード

例2-1: 成功（303）と失敗（400）を、テストで確かめる

> **▼ このコードがやること（先に日本語で）**：1つ目のテストでは、alice でログインしてノートを作り、①POST の返事（状態コード・移動先・お知らせのクッキー）、②移動先を GET したときの画面とクッキー、③もう一度 GET したとき（再読み込み）を、順に表示します。2つ目のテストでは、タイトルを空にして送り、返ってきた作成フォームの HTML を表示します。次に、タイトルを空白だけ・本文を 10,001 文字にして送り、エラーの文と入力が残っているかを表示します。どちらも、最後に `notes` 表の件数を数えます。

```rust
//! 第10巻 UNIT 02: 作成のフォーム（POST /notes）の、成功（303）と失敗（400）を確かめる。
//! 実行: cargo test --test u02_create -- --nocapture --test-threads=1

mod common;

use common::{TestApp, body_text, location, test_app};
use topcoat::router::{header, response::Response};

const PASSWORD: &str = "correct horse battery";

/// Set-Cookie の中から、お知らせ（flash=...）のクッキーを探す
fn flash_cookie(response: &Response) -> Option<String> {
    response
        .headers()
        .get_all(header::SET_COOKIE)
        .iter()
        .filter_map(|value| value.to_str().ok())
        .find(|value| value.starts_with("flash="))
        .map(str::to_string)
}

/// notes 表に何件あるか
async fn count_notes(app: &TestApp) -> i64 {
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query("SELECT COUNT(*) FROM notes", ())
        .await
        .expect("数えられません");
    let row = rows
        .next()
        .await
        .expect("行を読めません")
        .expect("行がない");
    row.get(0).expect("COUNT(*)")
}

#[tokio::test]
async fn success_redirects_with_303() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;

    // 1. フォームを送る（POST）
    let form =
"title=%E8%B2%B7%E3%81%84%E7%89%A9%E3%83%A1%E3%83%A2&body=%E7%89%9B%E4%B9%B3";
    let response = app.post_form("/notes", form, Some(&alice)).await;
    let to = location(&response).to_string();
    let flash = flash_cookie(&response).expect("お知らせのクッキーがない");
    println!("1. POST /notes → {}", response.status());
    println!("   Location: {to}");
```

```
    println!("   Set-Cookie: {flash}");

    // 2. ブラウザは Location へ移る（GET）。お知らせのクッキーも一緒に送る
    let cookie = format!("{alice}; flash=note_created");
    let response = app.get(&to, Some(&cookie)).await;
    let removed = flash_cookie(&response).unwrap_or_default();
    let html = body_text(response).await;
    println!("2. GET {to}");
    println!(
        "   お知らせ「ノートを作成しました」: {}",
        html.contains("ノートを作成しました")
    );
    println!(
        "   タイトル「買い物メモ」: {}",
        html.contains("<h1>買い物メモ</h1>")
    );
    println!("   Set-Cookie: {removed}");

    // 3. 再読み込み（同じ URL をもう一度 GET。お知らせのクッキーはもう無い）
    let html = body_text(app.get(&to, Some(&alice)).await).await;
    println!("3. もう一度 GET {to}（再読み込み）");
    println!(
        "   お知らせ「ノートを作成しました」: {}",
        html.contains("ノートを作成しました")
    );
    println!("notes 表の件数: {}", count_notes(&app).await);
}

#[tokio::test]
async fn invalid_input_is_shown_again_with_400() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;

    // タイトルを空にして、本文だけ書いて送る
    let response = app
        .post_form("/notes", "title=&body=%E7%89%9B%E4%B9%B3", Some(&alice))
        .await;
    println!("[タイトルが空] POST /notes → {}", response.status());
    println!(
        "   Location ヘッダー: {}",
        response.headers().contains_key(header::LOCATION)
    );
    let html = body_text(response).await;
    // 作成フォームの部分だけを抜き出して表示する
    let start = html
        .find(r#"<details class="new-note""#)
        .expect("フォームがない");
    let end = html[start..].find("</details>").expect("閉じタグがない") + start;
    println!(
        "{}",
```

```
        html[start..end + "</details>".len()].replace("><", ">\n<")
    );
    println!("notes 表の件数: {}", count_notes(&app).await);

    // タイトルは空白だけ、本文は 10,001 文字（両方とも不合格）
    let long_body = "あ".repeat(10_001);
    let form = format!("title=+++&body={}", "%E3%81%82".repeat(10_001));
    let response = app.post_form("/notes", &form, Some(&alice)).await;
    println!("[両方とも不合格] POST /notes → {}", response.status());
    let html = body_text(response).await;
    println!(
        "    「タイトルを入力してください」: {}",
        html.contains("タイトルを入力してください")
    );
    println!(
        "    「本文は10000文字以内にしてください（現在10001文字）」: {}",
        html.contains("本文は10000文字以内にしてください（現在10001文字）")
    );
    println!(
        "    本文の入力欄に 10,001 文字が残っている: {}",
        html.contains(&format!(">{long_body}</textarea>"))
    );
    println!("notes 表の件数: {}", count_notes(&app).await);
}
```

コピーした `rustnote` フォルダの `tests/u02_create.rs` として保存して実行します。

```
cargo test --test u02_create -- --nocapture --test-threads=1
```

▼ 実行結果（`POST /notes -> …` のような行は、RustNote のアクセスログです。Expires の日時は、実行した日によって変わります）

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.78s
     Running tests\u02_create.rs (target\debug\deps\u02_create-b3f494a9a5c39448.exe)

running 2 tests
test invalid_input_is_shown_again_with_400 ... POST /signup -> 303 See Other
(399.688ms)
POST /notes -> 400 Bad Request (3.5364ms)
[タイトルが空] POST /notes → 400 Bad Request
   Location ヘッダー: false
<details class="new-note" open="">
<summary>新しいノートを書く</summary>
<form method="post" action="/notes" class="note-form">
<label for="title">タイトル（80文字まで）</label>
<input id="title" name="title" value="" required="" maxlength="80">
<p class="field-error">タイトルを入力してください</p>
<label for="body">本文（10000文字まで）</label>
<textarea id="body" name="body" rows="6">牛乳</textarea>
<button type="submit">作成する</button>
</form>
</details>
notes 表の件数: 0
POST /notes -> 400 Bad Request (5.9791ms)
[両方とも不合格] POST /notes → 400 Bad Request
   「タイトルを入力してください」: true
   「本文は10000文字以内にしてください（現在10001文字）」: true
   本文の入力欄に 10,001 文字が残っている: true
notes 表の件数: 0
ok
test success_redirects_with_303 ... POST /signup -> 303 See Other (405.093ms)
POST /notes -> 303 See Other (7.377ms)
1. POST /notes → 303 See Other
   Location: /notes/1
   Set-Cookie: flash=note_created; HttpOnly; SameSite=Lax; Path=/; Max-Age=300
GET /notes/1 -> 200 OK (4.793ms)
2. GET /notes/1
   お知らせ「ノートを作成しました」: true
   タイトル「買い物メモ」: true
   Set-Cookie: flash=; Path=/; Max-Age=0; Expires=Wed, 29 Oct 2025 15:16:39 GMT
GET /notes/1 -> 200 OK (3.5781ms)
3. もう一度 GET /notes/1（再読み込み）
   お知らせ「ノートを作成しました」: false
notes 表の件数: 1
ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.90s
```

▼ コードを1つずつ分解して解説

解説1: 成功すると、画面ではなく 303 と移動先が返る

- `success_redirects_with_303` の1では、`303 See Other` と `Location: /notes/1` が返りました。画面の HTML はありません。ブラウザとサーバーのやり取りの図の②です。
- 一緒に `flash=note_created; HttpOnly; SameSite=Lax; Path=/; Max-Age=300` というクッキーが付きました。「移った先で『ノートを作成しました』を出してください」という予約です。`Max-Age=300` は 300 秒（5 分）で、移るのに失敗しても、いつまでも残らないようにしています。

解説2: お知らせは一度だけ出る

- 2で `/notes/1` を GET すると、お知らせとタイトル（`<h1>買い物メモ</h1>`）が出ました。同時に、`flash=; Path=/; Max-Age=0; Expires=…` で、お知らせのクッキーを消しています。`Max-Age=0` と、1年前の日時の `Expires` は、どちらも「このクッキーは、もう期限が切れています」という意味です。
- 3の再読み込みでは、お知らせは `false`（出ない）でした。`notes` 表の件数は 1 のままです。

解説3: 失敗すると、同じ画面が 400 で戻る

- `invalid_input_is_shown_again_with_400` では、`400 Bad Request` が返り、`Location` ヘッダーはありません（移らない）。`notes` 表は 0 件で、何も保存していません。
- 表示したフォームの HTML を見ると、次のことが分かります。
- `<details class="new-note" open="">`: 作成フォームは、ふだんは閉じた「新しいノートを書く」の中にありますが、エラーがあるときは `open` で**開いた**まま返します。閉じていたら、利用者はエラーに気づけません。
- `<p class="field-error">タイトルを入力してください</p>`: エラーは、その項目の**すぐ下**に出ます。
- `<textarea …>牛乳</textarea>`: 本文の「牛乳」は、消さずに残っています。
- 両方とも不合格のときは、タイトルのエラーと、本文のエラー（`本文は10000文字以内にしてください（現在10001文字）`）が**両方**出ました。10,001 文字の本文も、入力欄に残っています。1万文字を書いた人に「長すぎます」と言って、書いたものを消してしまったら、もう使ってもらえません。

もう一歩: 実務に近い形

例2-2: 再読み込みと、POST のやり直しを比べる

PRG で防げることと、防げないことを確かめます。

> **▼ このコードがやること（先に日本語で）**: ノートを1件作ってから、移動先の `/notes/1` を3回 GET し（再読み込みを3回）、`notes` 表の id を表示します。次に、同じフォームの POST をもう一度送り、もう一度 id を表示します。

```rust
//! 第10巻 UNIT 02: 「移った先を読み直す」と「POST をもう一度送る」で、ノートが何件になるかを比べる。
//! 実行: cargo test --test u02_reload -- --nocapture

mod common;

use common::{TestApp, location, test_app};

const PASSWORD: &str = "correct horse battery";
const FORM: &str = "title=%E8%B2%B7%E3%81%84%E7%89%A9%E3%83%A1%E3%83%A2&body=";

/// notes 表の id を、小さい順に読む
async fn note_ids(app: &TestApp) -> Vec<i64> {
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query("SELECT id FROM notes ORDER BY id", ())
        .await
        .expect("読めません");
    let mut ids = Vec::new();
    while let Some(row) = rows.next().await.expect("行を読めません") {
        ids.push(row.get(0).expect("id"));
    }
    ids
}

#[tokio::test]
async fn reload_after_redirect_does_not_duplicate() {
    let app = test_app().await;
    let alice = app.sign_up("alice@example.com", PASSWORD).await;

    let response = app.post_form("/notes", FORM, Some(&alice)).await;
    let to = location(&response).to_string();
    println!("POST /notes → {} (Location: {to}) ", response.status());
    for n in 1..=3 {
        let response = app.get(&to, Some(&alice)).await;
        println!("再読み込み {n} 回目: GET {to} → {}", response.status());
    }
    println!("ノートの id: {:?}", note_ids(&app).await);

    // もし POST の結果の画面にとどまっていたら、再読み込みは「POST をもう一度」になる
    let response = app.post_form("/notes", FORM, Some(&alice)).await;
    println!(
        "同じ POST をもう一度送る → {} (Location: {}) ",
        response.status(),
        location(&response)
    );
    println!("ノートの id: {:?}", note_ids(&app).await);
}
```

```
cargo test --test u02_reload -- --nocapture
```

▼ 実行結果

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.75s
     Running tests\u02_reload.rs (target\debug\deps\u02_reload-c30daf2536228748.exe)

running 1 test
POST /signup -> 303 See Other (388.9575ms)
POST /notes -> 303 See Other (6.7285ms)
POST /notes → 303 See Other（Location: /notes/1）
GET /notes/1 -> 200 OK (3.8365ms)
再読み込み 1 回目: GET /notes/1 → 200 OK
GET /notes/1 -> 200 OK (2.8887ms)
再読み込み 2 回目: GET /notes/1 → 200 OK
GET /notes/1 -> 200 OK (2.9527ms)
再読み込み 3 回目: GET /notes/1 → 200 OK
ノートの id: [1]
POST /notes -> 303 See Other (6.5562ms)
同じ POST をもう一度送る → 303 See Other（Location: /notes/2）
ノートの id: [1, 2]
test reload_after_redirect_does_not_duplicate ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.43s
```

- 再読み込みを3回しても、ノートの id は `[1]` のままです。くり返されたのは GET だけだからです。
- ところが、同じ POST をもう一度送ると、`/notes/2` ができ、id は `[1, 2]` になりました。RustNote は「同じ内容のノートを、少し前にも作った」かどうかを調べていません。
- もし作成のあとで、303 で移らずに、POST の返事として画面を返していたら、ブラウザの再読み込みは「POST をもう一度」になります（多くのブラウザは「フォームを再送信しますか」と聞きます）。そこで「はい」を押すと、ノートが2件になります。**PRG は、この再読み込みの二重登録を防ぐ形**です。
- 一方で、ボタンを素早く2回押した・通信が遅くて2回押した、という**二重送信**は、PRG では防げません。2回とも、本物の POST だからです。防ぐには、押したらボタンを押せなくする（ブラウザの中で動く部分が要る）や、フォームごとに一度しか使えない番号を入れておき、同じ番号の2回目は受け付けない（冪等性。第5巻 UNIT 08）といった方法があります。RustNote は、ノートが2件できても消せば済むので、ここまではしていません。

> **注意:** 303 と 302・307 の違い。303 は「次は GET で取りに来て」と決まっている番号です。307 は「同じメソッド（POST）のまま、送り直して」という意味なので、PRG には使いません。302 は、多くのブラウザが GET に変えますが、決まりの上では言い切れないので、POST のあとは 303 を使うのが確実です。

よくあるエラーと読み方

エラー1: String を、そのまま &str の場所に渡した（E0308）

`NoteForm` の `title` と `body` は `String` で、`NewNote::parse` は `&str` を受け取ります。`&` を付けずに渡した場合です（`examples/e02_string.rs`）。

```
// フォームの値（String）を、そのまま NewNote::parse に渡してしまった
use rustnote::domain::NewNote;

struct NoteForm {
    title: String,
    body: String,
}

fn main() {
    let input = NoteForm {
        title: "買い物メモ".to_string(),
        body: "牛乳".to_string(),
    };
    let note = NewNote::parse(input.title, input.body);
    println!("{note:?}");
}
```

```
cargo build --example e02_string
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0308]: arguments to this function are incorrect
   --> examples\e02_string.rs:14:16
    |
 14 |     let note = NewNote::parse(input.title, input.body);
    |                ^^^^^^^^^^^^^^ -----------  ---------- expected `&str`, found
`String`
    |                               |
    |                               expected `&str`, found `String`
    |
note: associated function defined here
   --> src\domain.rs:331:12
    |
331 |     pub fn parse(title: &str, body: &str) -> Result<NewNote, NoteInputErrors> {
    |            ^^^^^
help: consider borrowing here
    |
 14 |     let note = NewNote::parse(&input.title, input.body);
    |                               +
help: consider borrowing here
    |
 14 |     let note = NewNote::parse(input.title, &input.body);
    |                                            +

For more information about this error, try `rustc --explain E0308`.
error: could not compile `rustnote` (example "e02_string") due to 1 previous error
```

- **何が起きたか:** 「関数への引数が正しくありません」というエラーです。2つの引数のどちらにも `expected &str, found String`（`&str` が来るはずなのに、`String` が来た）が付いています。`note:` が、`parse(title: &str, body: &str)` の定義を指しています。
- **直し方:** `help:` のとおり、`&input.title` と `&input.body` のように `&` を付けて、**借りて**渡します（第3巻 UNIT 04）。`help:` は2つに分かれていますが、両方に `&` が要ります。RustNote の `create` も、`NewNote::parse(&input.title, &input.body)` と書いています。
- **学べること:** もし `parse` が `String` を受け取る形だったら、`input.title` の持ち主が `parse` に移ってしまい、あとで `NoteFormState::from_input(&input.title, …)` で入力を出し直すことができなくなります。検査する関数が `&str` を受け取るのは、**入力を手元に残したまま調べられる**ようにするためです。

エラー2: フォームの name と、NoteForm の項目名が違う

HTML の入力欄の `name` を、うっかり `subject` にしてしまった場合です。コンパイルは通り、画面も表示されます。送ったときに初めて分かります。

```rust
//! 第10巻 UNIT 02 のエラー2: フォームの name が、NoteForm の項目名と合っていないとき。
//! 実行: cargo test --test u02_err_name -- --nocapture

mod common;

use common::{body_text, test_app};

#[tokio::test]
async fn wrong_field_name() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;

    // HTML の入力欄を name="subject" にしてしまった（NoteForm の項目は title）
    let cases = [
        (
            "name が subject",
            "subject=%E8%B2%B7%E3%81%84%E7%89%A9&body=",
        ),
        ("本文の欄が無い", "title=%E8%B2%B7%E3%81%84%E7%89%A9"),
        (
            "余分な欄がある",
            "title=%E8%B2%B7%E3%81%84%E7%89%A9&body=&color=red",
        ),
    ];
    for (label, form) in cases {
        let response = app.post_form("/notes", form, Some(&alice)).await;
        let status = response.status();
        let html = body_text(response).await;
        println!(
            "[{label}] {status} / 「送信された内容を読み取れませんでした」: {}",
            html.contains("送信された内容を読み取れませんでした")
        );
    }
}
```

```
cargo test --test u02_err_name -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.47s
     Running tests\u02_err_name.rs (target\debug\deps\u02_err_name-
9c4aff559b955ba0.exe)

running 1 test
POST /signup -> 303 See Other (387.4675ms)
POST /notes -> 400 Bad Request (2.1269ms)
[name が subject] 400 Bad Request / 「送信された内容を読み取れませんでした」: true
POST /notes -> 400 Bad Request (1.8745ms)
[本文の欄が無い] 400 Bad Request / 「送信された内容を読み取れませんでした」: true
POST /notes -> 303 See Other (5.9389ms)
[余分な欄がある] 303 See Other / 「送信された内容を読み取れませんでした」: false
test wrong_field_name ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.41s
```

- **何が起きたか:** `name` が `subject` のときと、本文の欄そのものが無いときは、`400 Bad Request` と「送信された内容を読み取れませんでした」になりました。Topcoat の `Form<NoteForm>` が、送られてきた値を `NoteForm` に変えようとして、`title`（または `body`）が見つからなかったからです。この 400 は、`create` の中のコードが動く**前**に返るので、項目ごとのエラーも、入力の出し直しもありません。サーバーのログにも、アクセスログの1行のほかは何も残りません。
- **余分な欄は、黙って無視されます。** `color=red` を足しても 303 で作成できました。`NoteForm` に無い項目は、読まずに捨てられます。
- **直し方:** HTML の `name="…"` と、`NoteForm` の項目名をそろえます。RustNote の `note_form` は `name="title"` と `name="body"` です。
- **学べること:** HTML と Rust の構造体の名前は、コンパイラーが照らし合わせてくれません。例2-1のように、**本物のフォームと同じ形の文字列を POST するテスト**を残しておくと、どちらかの名前を変えたときに、テストが気づかせてくれます（第11巻 UNIT 03（準備中））。

RustNote ではここで使う

作成の関数 `create` です（`src/routes/notes.rs` から抜き出しています）。

```
#[page(POST)]
pub async fn create(cx: &Cx, Form(input): Form<NoteForm>) -> Result<impl View> {
    let user = require_user(cx).await?;

    let errors = match NewNote::parse(&input.title, &input.body) {
        Ok(new_note) => {
            let note = app_state(cx)
                .notes
                .insert(user.id, &new_note)
                .await
                .map_err(AppError::from)?;
            flash::set(cx, Flash::NoteCreated);
            // PRG: 作成できたら 303 で詳細ページ（GET）へ
            return Err(see_other(href!(id::show, id::Id(note.id)).resolve(cx)).into());
        }
        Err(errors) => errors,
    };

    // 入力に問題があった: 400 で一覧と入力済みのフォームを再表示する
    let form = NoteFormState::from_input(&input.title,
&input.body).with_errors(&errors);
    Ok(view! {
        (StatusCode::BAD_REQUEST)
        notes_index(owner: user.id, query: NoteQuery::new(None, None), form: form)
    })
}
```

- `#[page(POST)]` で、`/notes` への POST をこの関数が受け取ります（同じ URL の GET は `index`）。`Form(input): Form<NoteForm>` が、送られてきたフォームを `NoteForm` に変えます。ここで失敗するのが、エラー2の 400 です。
- 成功したときの 303 は、`return Err(see_other(…).into())` と、**エラーの形**で返しています。Topcoat の `see_other` は `router::error` にある「エラーの形の返事」で、`Err` で返すと、画面を作るのをやめて、303 の返事になります（第6巻 UNIT 06）。`href!(id::show, id::Id(note.id))` は、詳細ページの URL（`/notes/1`）を、型を確かめながら作ります。
- 失敗したときは、`(StatusCode::BAD_REQUEST)` で状態コードを 400 にしてから、一覧の画面（`notes_index`）を、入力済みのフォームと一緒に作り直します。一覧の部分も一緒に出すので、利用者はもとの画面のまま、エラーを直せます。

検査の関数 `NewNote::parse` です（`src/domain.rs` から抜き出しています）。

```
impl NewNote {
    /// タイトルと本文をまとめて検査する。
    /// 両方の問題を一度に画面へ出せるよう、最初の失敗で止めずに全部集める。
    pub fn parse(title: &str, body: &str) -> Result<NewNote, NoteInputErrors> {
        match (NoteTitle::parse(title), NoteBody::parse(body)) {
            (Ok(title), Ok(body)) => Ok(NewNote { title, body }),
            (title, body) => Err(NoteInputErrors {
                title: title.err(),
                body: body.err(),
            }),
        }
    }
}
```

- 2つの検査を**両方とも先に**行い、結果の組 `(title, body)` で場合を分けます。4つの結果の図のうち、両方 `Ok` のときだけ `NewNote` を作ります。
- それ以外は、`title.err()`（`Err` なら `Some(エラー)`、`Ok` なら `None`）で、見つかった問題だけを `NoteInputErrors` に入れます。`?` で最初のエラーで止めてしまうと、タイトルを直して出し直したあとで、初めて本文のエラーを言われることになります。
- 合格した値は、`NoteTitle`・`NoteBody` という型になります。`notes.insert` は `NewNote` しか受け取らないので、**検査していない値を保存する道は、コンパイラーが閉じています**（第4巻 UNIT 01）。

入力を残すのは、`NoteFormState` です（`src/views.rs` から抜き出しています）。

```
impl NoteFormState {
    /// 入力された値をそのまま入れた状態（エラーで再表示するとき、入力を消さないため）
    pub fn from_input(title: &str, body: &str) -> NoteFormState {
        NoteFormState {
            title: title.to_string(),
            body: body.to_string(),
            ..NoteFormState::default()
        }
    }

    /// 検査で見つかった問題を、項目ごとの文章にして入れる
    pub fn with_errors(mut self, errors: &NoteInputErrors) -> NoteFormState {
        self.title_error = errors.title.map(|e| e.to_string());
        self.body_error = errors.body.map(|e| e.to_string());
        self
    }
}
```

- `from_input` には、検査する**前**の、送られてきたままの値を入れます。前後の空白を取る前の値なので、利用者が書いたとおりに戻ります。

- `with_errors` は、`TitleError` などを `to_string()` で文章（「タイトルを入力してください」）に変えます。文章は、`domain.rs` の `#[error("…")]` に書いてあります。

練習問題

問1: タイトルを `" 買い物 "`（前後に空白）にして作ると、保存されるタイトルはどうなるでしょうか。81 文字のタイトルを送ったときの、状態コードとエラーの文、入力欄に残る値も確かめてください。

問2: 本文に `"1行目\r\n2行目\r\n"` を入れて作り、`notes` 表に保存された本文と、その文字数を表示してください。ブラウザのフォームは、改行を `\r\n`（2文字）で送ってきます。

問3: タイトルに `<b>"大事"</b>` を入れたとき、(a) 本文が長すぎて 400 で出し直したときのタイトルの入力欄と、(b) 作成できたときの詳細ページの見出しが、それぞれ HTML の中でどう書かれているかを表示してください。本物の `<b>` タグとして、太字になってしまうでしょうか。

解答と解説

問1の解答:

`tests/u02_q1.rs` です。

```rust
//! 問1の解答: タイトルの前後の空白と、81 文字のタイトルを送ったときを確かめる。
//! 実行: cargo test --test u02_q1 -- --nocapture

mod common;

use common::{TestApp, body_text, encode, test_app};

/// notes 表のタイトルを、id の順に読む
async fn titles(app: &TestApp) -> Vec<String> {
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query("SELECT title FROM notes ORDER BY id", ())
        .await
        .expect("読めません");
    let mut titles = Vec::new();
    while let Some(row) = rows.next().await.expect("行を読めません") {
        titles.push(row.get(0).expect("title"));
    }
    titles
}

#[tokio::test]
async fn title_is_trimmed_and_limited() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;

    let form = format!("title={}&body=", encode("  買い物  "));
    let response = app.post_form("/notes", &form, Some(&alice)).await;
    println!("[前後に空白] {}", response.status());
    println!("   保存されたタイトル: {:?}", titles(&app).await);

    let long_title = "あ".repeat(81);
    let form = format!("title={}&body=", encode(&long_title));
    let response = app.post_form("/notes", &form, Some(&alice)).await;
    let status = response.status();
    let html = body_text(response).await;
    println!("[81 文字] {status}");
    println!(
        "    「タイトルは80文字以内にしてください（現在81文字）」: {}",
        html.contains("タイトルは80文字以内にしてください（現在81文字）")
    );
    println!(
        "   入力欄に 81 文字が残っている: {}",
        html.contains(&format!(r#"value="{long_title}""#))
    );
```

```
    println!("   保存されたタイトル: {:?}", titles(&app).await);
}
```

```
cargo test --test u02_q1 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.73s
     Running tests\u02_q1.rs (target\debug\deps\u02_q1-c95d3e8f58bcc743.exe)

running 1 test
POST /signup -> 303 See Other (386.115ms)
POST /notes -> 303 See Other (6.0805ms)
[前後に空白] 303 See Other
   保存されたタイトル: ["買い物"]
POST /notes -> 400 Bad Request (4.1825ms)
[81 文字] 400 Bad Request
   「タイトルは80文字以内にしてください（現在81文字）」: true
   入力欄に 81 文字が残っている: true
   保存されたタイトル: ["買い物"]
test title_is_trimmed_and_limited ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.42s
```

- 前後に空白があるタイトルは、`"買い物"` として保存されました。`NoteTitle::parse` が `trim()` で前後の空白を取ってから、文字数を数えて保存するからです。
- 81 文字のタイトルは 400 で、「タイトルは80文字以内にしてください（現在81文字）」が出ました。今の文字数も出すので、あと何文字減らせばよいかが分かります。入力欄には 81 文字がそのまま残り、保存されたタイトルは増えていません。
- 入力欄には `maxlength="80"` も付いているので、ブラウザは、80 を超えて打ち込めないようにします（HTML の決まりでは UTF-16 の単位で数えるので、絵文字などでは、サーバーの数え方とずれます。第11巻 UNIT 01）。それでもサーバーで検査するのは、ブラウザの決まりは、テストのように直接 POST すれば、いくらでも外せるからです。**ブラウザの検査は使いやすさのため、サーバーの検査は守りのため**です。

問2の解答:

`tests/u02_q2.rs` です。

```rust
//! 問2の解答: 本文の改行（ブラウザは \r\n で送る）が、どう保存されるかを確かめる。
//! 実行: cargo test --test u02_q2 -- --nocapture

mod common;

use common::{encode, test_app};

#[tokio::test]
async fn newlines_are_normalized() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;

    let sent = "1行目\r\n2行目\r\n";
    let form = format!("title={}&body={}", encode("改行"), encode(sent));
    println!("送ったフォーム: {form}");
    app.create_note(&alice, "改行", sent).await;

    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query("SELECT body, length(body) FROM notes", ())
        .await
        .expect("読めません");
    let row = rows
        .next()
        .await
        .expect("行を読めません")
        .expect("行がない");
    let body: String = row.get(0).expect("body");
    let length: i64 = row.get(1).expect("length");
    println!("送った本文: {sent:?}（{} 文字）", sent.chars().count());
    println!("保存された本文: {body:?}（SQLite の length: {length}）");
}
```

```
cargo test --test u02_q2 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.84s
     Running tests\u02_q2.rs (target\debug\deps\u02_q2-6804b1bc9d1c3136.exe)

running 1 test
POST /signup -> 303 See Other (394.3234ms)
送ったフォーム:
title=%E6%94%B9%E8%A1%8C&body=1%E8%A1%8C%E7%9B%AE%0D%0A2%E8%A1%8C%E7%9B%AE%0D%0A
POST /notes -> 303 See Other (7.528ms)
送った本文: "1行目\r\n2行目\r\n"（10 文字）
保存された本文: "1行目\n2行目\n"（SQLite の length: 8）
test newlines_are_normalized ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.53s
```

- 送った本文は 10 文字（`\r` と `\n` を1文字ずつ数える）でしたが、保存された本文は `"1行目\n2行目\n"` の 8 文字でした。`NoteBody::parse` が、`\r\n` を `\n` にそろえてから保存するからです。フォームの中の `%0D%0A` が、`\r\n` です。
- そろえるのは、文字数を数える**前**です。そろえないと、改行が 1,000 個ある本文は、見た目より 1,000 文字多く数えられて、「10,000 文字まで」の決まりに早く引っかかってしまいます。保存する形がそろっていれば、検索（UNIT 07（準備中））や比べるときにも困りません。

問3の解答:

`tests/u02_q3.rs` です。

```rust
//! 問3の解答: タイトルに HTML のタグや引用符を入れたとき、画面にどう出るかを確かめる。
//! 実行: cargo test --test u02_q3 -- --nocapture

mod common;

use common::{body_text, encode, test_app};

/// HTML の中から、start で始まり end で終わる最初の部分を抜き出す
fn between<'a>(html: &'a str, start: &str, end: &str) -> &'a str {
    let from = html.find(start).expect("始まりが見つかりません");
    let to = html[from..].find(end).expect("終わりが見つかりません") + from;
    &html[from..to + end.len()]
}

#[tokio::test]
async fn title_is_escaped() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;
    let title = r#"<b>"大事"</b>"#;

    // 失敗して出し直すとき: 本文を長すぎるようにして、400 にする
    let form = format!(
        "title={}&body={}",
        encode(title),
        encode(&"あ".repeat(10_001))
    );
    let response = app.post_form("/notes", &form, Some(&alice)).await;
    let status = response.status();
    let html = body_text(response).await;
    println!("[400 で出し直す] {status}");
    println!(
        "   {}",
        between(&html, r#"<input id="title""#, r#"maxlength="80">"#)
    );
    println!(
        "   \" がそのまま入っている（value から抜け出せる）: {}",
        html.contains(r#"value="<b>"大事""#)
    );

    // 作成できたとき: 詳細ページの見出し
    let to = app.create_note(&alice, title, "").await;
    let html = body_text(app.get(&to, Some(&alice)).await).await;
    println!("[詳細ページ] {}", between(&html, "<h1>", "</h1>"));
    println!(
        "   本物の <b> タグになっている: {}",
        html.contains("<h1><b>")
```

```
    );
}
```

```
cargo test --test u02_q3 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.49s
     Running tests\u02_q3.rs (target\debug\deps\u02_q3-91faee38663bf341.exe)

running 1 test
POST /signup -> 303 See Other (417.1372ms)
POST /notes -> 400 Bad Request (7.3145ms)
[400 で出し直す] 400 Bad Request
   <input id="title" name="title" value="<b>&quot;大事&quot;</b>" required=""
maxlength="80">
   " がそのまま入っている（value から抜け出せる）: false
POST /notes -> 303 See Other (5.1299ms)
GET /notes/1 -> 200 OK (5.8205ms)
[詳細ページ] <h1>&lt;b&gt;"大事"&lt;/b&gt;</h1>
   本物の <b> タグになっている: false
test title_is_escaped ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.46s
```

- どちらも、本物の `<b>` タグにはなりませんでした。Topcoat の `view!` は、値を入れる**場所に合わせて**、危ない文字を書き換えます（エスケープ）。
- (a) 入力欄の `value="…"` の中では、`"` が `"` になりました。`"` がそのままだと、そこで `value` が終わってしまい、うしろに好きな属性を書かれてしまうからです。`value="…"` の中の `<` と `>` は、タグの始まりにはならないので、そのままです。
- (b) 見出しの中では、`<` と `>` が `<` と `>` になりました。ここでは `<` がタグの始まりになるからです。`"` は、ここでは何も起こさないので、そのままです。
- ブラウザには、どちらも `<b>"大事"</b>` という**文字**として表示されます。もし、ここに `<script>` を書かれて、それがそのまま HTML に入ると、ほかの人の画面で好きなプログラムを動かされてしまいます（XSS）。RustNote では、ノートは持ち主しか見られませんが、それでも、利用者の入力は**必ずエスケープして**画面に出します。エスケープの仕組みは、第6巻 UNIT 05でくわしく扱います。

まとめ

- 作成は、ログインの確かめ → `NewNote::parse` で入力の検査 → 合格なら保存して 303、不合格なら保存せずに 400 で同じ画面を出し直す。
- 出し直すときは、送られてきたままの入力を残し、項目ごとのエラーを全部まとめて、その項目のすぐ下に出す。フォームは開いたまま返す。

- PRG で、再読み込みは GET のくり返しになり、ノートは二重にできない。ただし、POST そのものの二重送信は PRG では防げない。フォームの `name` と `NoteForm` の項目名がずれると、コードが動く前に 400 になるので、本物と同じ形の POST をテストに残す。

この巻のまとめ

この版に収録した UNIT 01〜02 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: 一覧・空のとき・持ち主の条件

- ☐ `GET /notes` が、`require_user` → `NoteQuery::new` → `notes.search` → `notes_index`（`view!`）の順で作られていることを説明できる
- ☐ 一覧の SQL の `user_id = ?1`・`ORDER BY updated_at DESC, id DESC`・`LIMIT 20 OFFSET …` の、それぞれの理由を言える
- ☐ 同じ時刻のノートの順番は SQL では決まっていないことと、試して同じでも `id` を足す理由を説明できる
- ☐ 0 件のときと、検索で 0 件のときで、出す文を分ける理由を言える
- ☐ `view!` に `( )` で入れた値が自動でエスケープされることを、テストで確かめられる
- ☐ `for note in page.notes` で起きる `E0382` と、`help` どおりに `&` を付けたときの `E0507` を読み、`clone()` で直せる
- ☐ 件数の SQL にだけ持ち主の条件を忘れると、件数から他人のノートの数が分かってしまうことを説明できる

UNIT 02: 作成のフォーム・入力の検査・PRG

- ☐ `POST /notes` が、`require_user` → `NewNote::parse` → 保存して 303、または保存せずに 400、と分かれることを説明できる
- ☐ 成功の返事が `303 See Other` と `Location` とお知らせのクッキーだけで、画面は GET で取りに行くこと（PRG）を、テストで確かめられる
- ☐ 再読み込みではノートが増えないのに、同じ POST をもう一度送ると増えることと、二重送信の防ぎ方を言える
- ☐ 400 で出し直すとき、入力を残し、項目ごとのエラーを全部まとめ、フォームを開いたまま返す理由を説明できる
- ☐ `NewNote::parse` が、両方の検査をしてから結果をまとめる理由を言える
- ☐ `String` を `&str` の場所に渡したときの `E0308` と、フォームの `name` が違うときの 400 を読める
- ☐ 前後の空白・改行の `\r\n`・タグや引用符の入ったタイトルが、どう保存され、どう表示されるかを確かめられる

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03（詳細・URL の ID・見つからないとき）・UNIT 04（編集・入力済みのフォーム・競合）・UNIT 05（完了の切り替え・1文の SQL）・UNIT 06（削除・確かめ・消し方の選び方）・UNIT 07（検索・LIKE のエスケープ）・UNIT 08（ページ送り・OFFSET と cursor）・UNIT 09（お知らせ・エラーのまとめ）・UNIT 10（狭い画面・アクセシビリティ・総合の試験）は準備中で、次の更新から順に追加します。それまでの間、それぞれの画面のコードは、`Rust/rustnote/src/routes/notes.rs` と `src/routes/notes/` の各ファイルの先頭のコメントで確かめられます。

用語ミニ辞典

この巻（UNIT 01〜02）に出てきた用語です。

UNIT 01 の用語

用語	意味
CRUD	Create（作る）・Read（読む）・Update（直す）・Delete（消す）。データを扱う画面の基本の4つの操作
空の状態（empty state）	表示するデータが1件も無いときの画面。次にすることを伝える文を出す
持ち主の条件	SQL の `WHERE user_id = ?1`。ログイン中の人のデータだけを読む
`ORDER BY … DESC`	大きい（新しい）順に並べる指定。同じ値の行の順番は、決まりでは決まっていない
`LIMIT` / `OFFSET`	読む行の数／先頭から飛ばす行の数。RustNote は 1 ページ 20 件
`NoteQuery`	URL の `?q=`（キーワード）と `?page=`（ページ番号）を検査して入れる型
`#[component]`	`view!` の部品を作る印。RustNote の `notes_index` は、中で DB を読む async の部品
エスケープ	HTML で特別な意味を持つ文字（`<`・`>`・`&`）を、ただの文字の書き方（`<` など）に置き換えること
`class!`	条件に合うときだけクラスを足して、`class` 属性の値を作るマクロ
E0382	移動した（持ち主が変わった）値を、もう一度使おうとしたエラー
E0507	借りているだけの値から、中身を移動しようとしたエラー

UNIT 02 の用語

用語	意味
PRG（Post/Redirect/Get）	POST が成功したら、画面を返さずに 303 で別の URL へ移し、ブラウザに GET で開き直させる形
303 See Other	「次は、この URL を GET で取りに来て」という返事。`Location` ヘッダーに移動先が入る
400 Bad Request	送られてきた内容に問題がある、という返事。RustNote は、入力を残した画面と一緒に返す
入力の検査（バリデーション）	送られてきた値が決まりに合っているかを確かめること。合わなければ保存しない
`NoteInputErrors`	作成・編集の検査で見つかった問題を、項目ごと（`title`・`body`）に入れる型
`NoteFormState`	フォームに出す値とエラー。`from_input` で入力を残し、`with_errors` でエラーの文を入れる
お知らせ（フラッシュ）	移った先で一度だけ出すメッセージ。RustNote はクッキー `flash` で受け渡す
二重送信	同じフォームの POST が2回届くこと。PRG では防げない
冪等性（べきとうせい）	同じ操作を何回くり返しても、1回したときと結果が変わらない性質
`"`	HTML の属性の値の中で、`"` を、ただの文字として書く書き方
XSS	利用者の入力に書かれたプログラムが、ほかの人の画面で動いてしまう弱さ
E0308	型が合わないエラー（ここでは `&str` の場所に `String` を渡した）